3D WEBSOFT INC. FOUNDATION

FUTURE TECH SCHOOL PROGRAM

Faculty Name :

Course Duration : 5 Months

Classes : Offline & Online

Daily Teaching : 2 Hours

Total Classes : 100

Level 4 – AI Professional Foundation(11th–12th Standard)

*Class 10: *args and **kwargs – Flexible Functions*

3DWEBSOFT FOUNDATION

Learning Objectives

By the end of this lesson, students will be able to:

- Understand why flexible functions are needed.
- Explain *args and **kwargs.
- Create functions that accept any number of arguments.
- Use positional and keyword arguments together.
- Apply *args and **kwargs in real-world Python programs.

1. What is a Flexible Function?

A **flexible function** is a function that can accept a **variable number of arguments**.

Normally, we define a function with a fixed number of parameters:

```
def add(a, b):
    return a + b
```

print(add(10, 20))

Here, the function expects exactly **2 arguments**.

But what if we want to add 2 numbers, 5 numbers, or 10 numbers?

We can use *args.

2. Understanding *args

*args allows a function to accept **any number of positional arguments**.

Syntax

```
def function_name(*args):
    # statements
```

Inside the function, args behaves like a **tuple**.

Example

```
def show_numbers(*args):
    print(args)

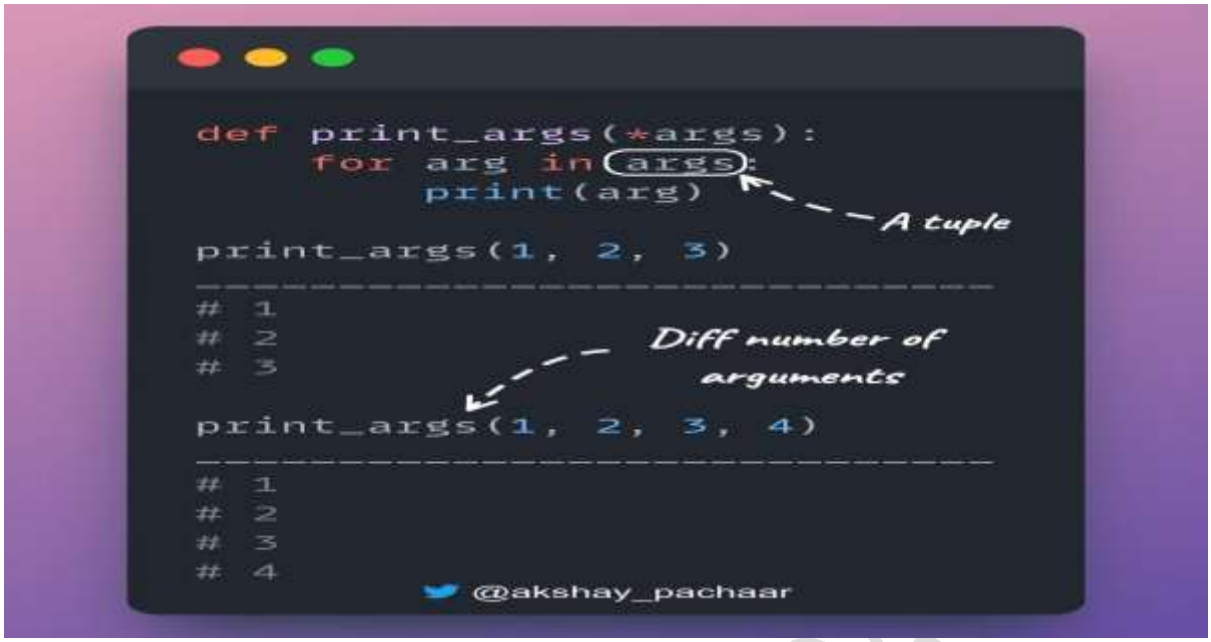
show_numbers(10, 20, 30)
```

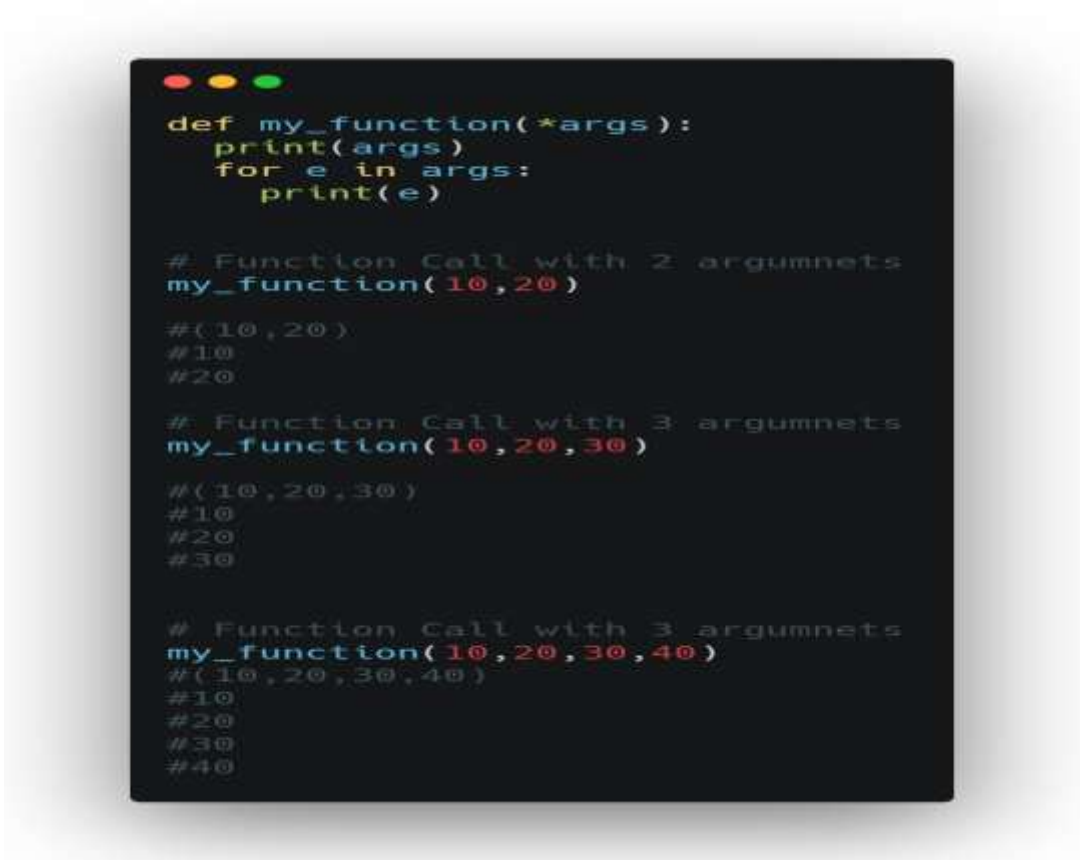
Output

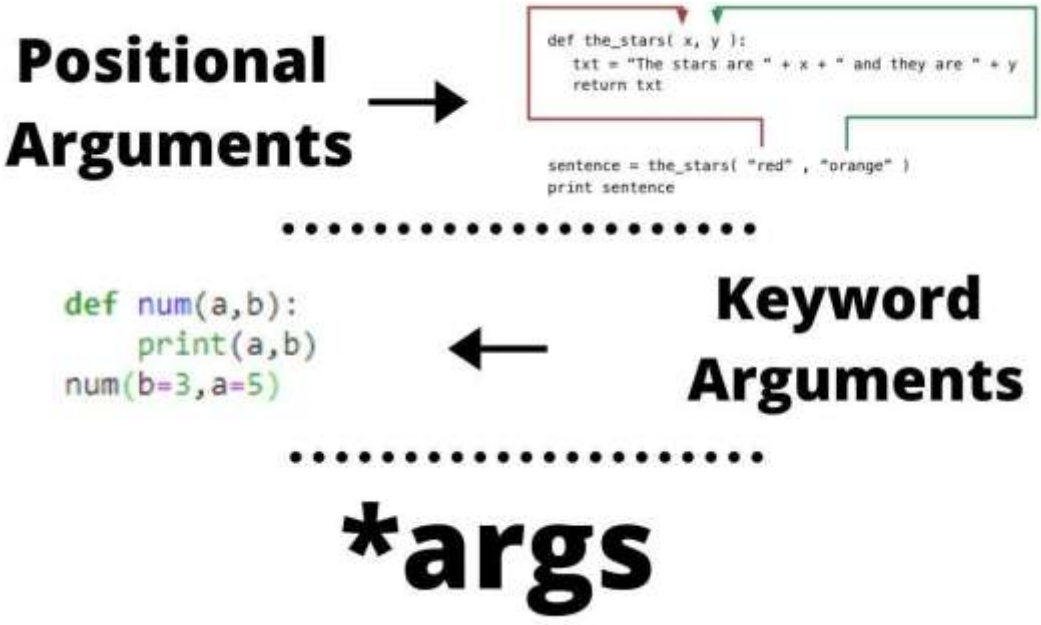
```
(10, 20, 30)
```

Notice that Python automatically stores the arguments inside a tuple.

3DWEBSOFT FOUNDATION


def print_args(*args):
for arg in args:
print(arg)
A tuple
print_args(1, 2, 3)
1
2
3
Diff number of arguments
print_args(1, 2, 3, 4)
1
2
3
4
@akshay_pachaar


def my_function(*args):
print(args)
for e in args:
print(e)
Function Call with 2 argumnets
my_function(10,20)
#(10,20)
#10
#20
Function Call with 3 argumnets
my_function(10,20,30)
#(10,20,30)
#10
#20
#30
Function Call with 3 argumnets
my_function(10,20,30,40)
#(10,20,30,40)
#10
#20
#30
#40



3. Why Do We Use *args?

Suppose we want to create an addition function.

Without *args:

```
def add(a, b):
    return a + b
```

We can only provide two numbers.

With *args:

```
def add(*args):
    total = 0

    for number in args:
        total += number

    return total
```

3DWEBSOFT FOUND

```
print(add(10, 20))
print(add(10, 20, 30))
print(add(10, 20, 30, 40, 50))
```

Output

```
30
60
150
```

The same function works with different numbers of arguments.

4. Important Point About *args

The name args is **not compulsory**.

The * is important.

For example:

```
def test(*numbers):
    print(numbers)

test(10, 20, 30)
```

This also works.

However, programmers commonly use the standard name:

*args

5. Accessing Values from args

Because args is a tuple, we can use indexing and loops.

```
def students(*names):
    print("First student:", names[0])
```

3DWEBSOFT FOUNDATION

```
students("Asha", "Rahul", "Kiran")
```

Output

```
First student: Asha
```

Using a loop

```
def students(*names):
    for name in names:
        print(name)

students("Asha", "Rahul", "Kiran")
```

Output:

```
Asha
Rahul
Kiran
```

6. Practical Example – Student Marks

```
def total_marks(*marks):
    total = 0

    for mark in marks:
        total += mark

    return total

print("Total:", total_marks(80, 75, 90, 85, 95))
```

Output:

```
Total: 425
```

3DWEBSOFT FOUNDATION

Average Marks

```
def average_marks(*marks):
    total = sum(marks)
    average = total / len(marks)
    return average

print(average_marks(80, 70, 90, 85))
```

Output:

81.25

7. What is **kwargs?

kwargs allows a function to accept **any number of keyword arguments.

Syntax

```
def function_name(**kwargs):
    # statements
```

Inside the function, kwargs behaves like a **dictionary**.

Example:

```
def student_info(**kwargs):
    print(kwargs)

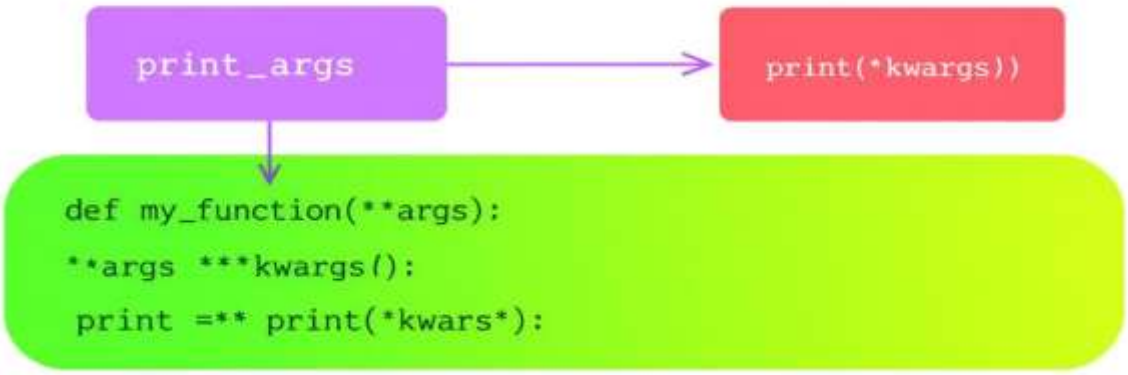
student_info(name="Rahul", age=17, course="Python")
```

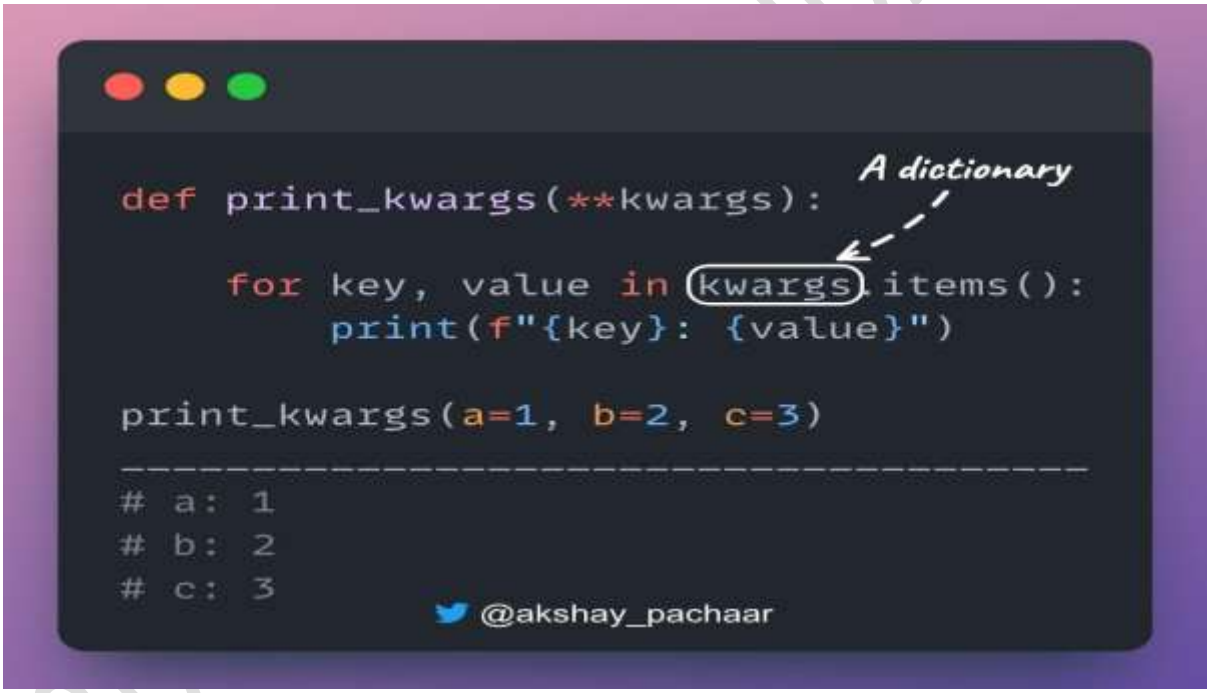
Output:

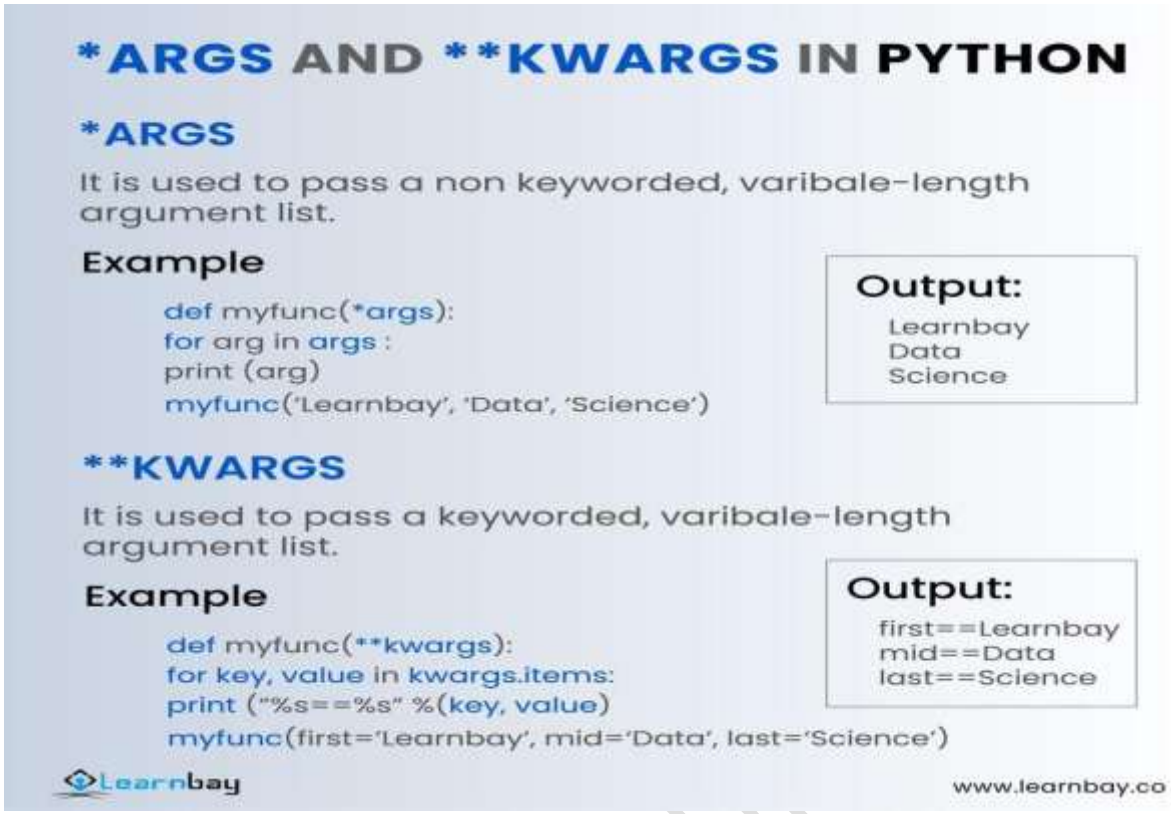
{'name': 'Rahul', 'age': 17, 'course': 'Python'}

3DWEBSOFT FOUNDATION

UNDERSTANDING PYTHON'S **args and *kwargs*







8. Accessing **kwargs

Since kwargs is a dictionary, we can access values using keys.

```
def student_info(**kwargs):
    print("Name:", kwargs["name"])
    print("Age:", kwargs["age"])
    print("Course:", kwargs["course"])

student_info(
    name="Rahul",
    age=17,
    course="Python"
)
```

Output:

3DWEBSOFT FOU

Name: Rahul

Age: 17

Course: Python

9. Using a Loop with **kwargs

```
def student_info(**details):
    for key, value in details.items():
        print(key, ":", value)

student_info(
    name="Anjali",
    age=16,
    city="Hyderabad",
    course="AI"
)
```

Output:

name : Anjali

age : 16

city : Hyderabad

course : AI

*10. Difference Between *args and **kwargs*

Feature	*args	**kwargs
Purpose	Multiple positional arguments	Multiple keyword arguments
Internal type	Tuple	Dictionary

3DWEBSOFT FOUNDATION

Feature	*args	**kwargs
Uses	*	**
Example	10, 20, 30	name="Rahul"
Access	Index	Key
Common use	Numbers/items	Named information

Easy Memory Trick

***args → many values → Tuple**

****kwargs → many named values → Dictionary**

*11. *args and **kwargs Together*

We can use both in the same function.

```
def student_data(*subjects, **details):
    print("Subjects:", subjects)
    print("Details:", details)

student_data(
    "Python",
    "AI",
    "Math",
    name="Anjali",
    age=17
)
```

Output:

```
Subjects: ('Python', 'AI', 'Math')
Details: {'name': 'Anjali', 'age': 17}
```

3DWEBSOFT FOUNDATION

Here:

- *subjects stores positional arguments.
- **details stores keyword arguments.

12. Function Argument Order

When using different types of parameters, a common order is:

```
def function(normal, *args, **kwargs):
    pass
```

Example:

```
def demo(name, *subjects, **details):
    print("Name:", name)
    print("Subjects:", subjects)
    print("Details:", details)

demo(
    "Rahul",
    "Python",
    "AI",
    age=17,
    city="Hyderabad"
)
```

13. Practical Project – Student Information System

Let's create a small program using **kwargs.

```
def display_student(**student):
    print("\n--- Student Details ---")
```

3DWEBSOFT FOUNDATION

```
    for key, value in student.items():
        print(key, ":", value)

display_student(
    name="Aisha",
    age=17,
    class_name="12th",
    course="AI & Python",
    city="Hyderabad"
)
```

Output:

```
--- Student Details ---
name : Aisha
age : 17
class_name : 12th
course : AI & Python
city : Hyderabad
```

14. Practical Project – Flexible Calculator

Let's create a calculator that can add any number of values.

```
def add_numbers(*numbers):
    return sum(numbers)

print("Result:", add_numbers(10, 20, 30, 40))
```

Output:

3DWEBSOFT FOUNDATION

Result: 100

We can also create multiplication:

```
def multiply_numbers(*numbers):
    result = 1

    for number in numbers:
        result *= number

    return result

print(multiply_numbers(2, 3, 4))
```

Output:

24

15. Real-World Example

Imagine an online shopping system.

Different customers may provide different information:

```
def customer_details(**customer):
    for key, value in customer.items():
        print(key, ":", value)

customer_details(
    name="Rahul",
    city="Hyderabad",
    phone="9999999999",
    membership="Gold"
```

3DWEBSOFT FOUNDATION

)

Another customer may provide additional information:

```
customer_details(
    name="Asha",
    city="Bengaluru",
    membership="Silver",
    discount=20
)
```

The function doesn't need to be rewritten.

That's why **kwargs is called **flexible**.

16. Unpacking with *

We can also use * to unpack a list or tuple.

```
numbers = [10, 20, 30]

def add(a, b, c):
    return a + b + c

print(add(*numbers))
```

Output:

60

Here:

*numbers

unpacks the list into:

10, 20, 30

3DWEBSOFT FOUNDATION

17. Unpacking with **

A dictionary can be unpacked using **.

```
student = {
   "name": "Rahul",
   "age": 17
}

def display(name, age):
   print("Name:", name)
   print("Age:", age)

display(**student)
```

Output:

Name: Rahul

Age: 17

18. Common Mistakes

Mistake 1: Forgetting *

Incorrect:

```
def add(args):
   print(args)
```

This accepts only one argument.

Correct:

```
def add(*args):
   print(args)
```

3DWEBSOFT FOUNDATION

Mistake 2: Forgetting **

Incorrect:

```
def student(kwargs):
    print(kwargs)
```

Correct:

```
def student(**kwargs):
    print(kwargs)
```

Mistake 3: Treating args as a dictionary

```
def test(*args):
    print(args["name"])
```

This is incorrect because args is a tuple.

Use indexing:

```
def test(*args):
    print(args[0])
```

Mistake 4: Treating kwargs as a tuple

```
def test(**kwargs):
    print(kwargs[0])
```

This is incorrect.

Use a dictionary key:

```
def test(**kwargs):
    print(kwargs["name"])
```

19. Classroom Activity

Ask students to create the following functions.

3DWEBSOFT FOUNDATION

Activity 1 – Sum

Create:

sum_numbers(*numbers)

Example:

sum_numbers(10, 20, 30, 40)

Expected output:

100

Activity 2 – Student Details

Create:

student(**details)

Call:

```
student(
   name="Rahul",
   age=17,
   grade="12th",
   city="Hyderabad"
)
```

Activity 3 – Shopping Cart

Create:

shopping_cart(*items)

Example:

shopping_cart("Laptop", "Mouse", "Keyboard", "Headphones")

Print every item.

20. Mini Project – Flexible Student Report

```
def student_report(name, *marks, **details):
```

3DWEBSOFT FOUNDATION

```
    print("\n===== STUDENT REPORT =====")

    print("Name:", name)

    print("\nMarks:")
    for mark in marks:
        print(mark)

    print("\nAdditional Details:")
    for key, value in details.items():
        print(key, ":", value)

student_report(
    "Rahul",
    85,
    90,
    78,
    92,
    class_name="12th",
    section="A",
    city="Hyderabad"
)
```

Expected Output

```
===== STUDENT REPORT =====
```

3DWEBSOFT FOUNDATION

Name: Rahul

Marks:

85

90

78

92

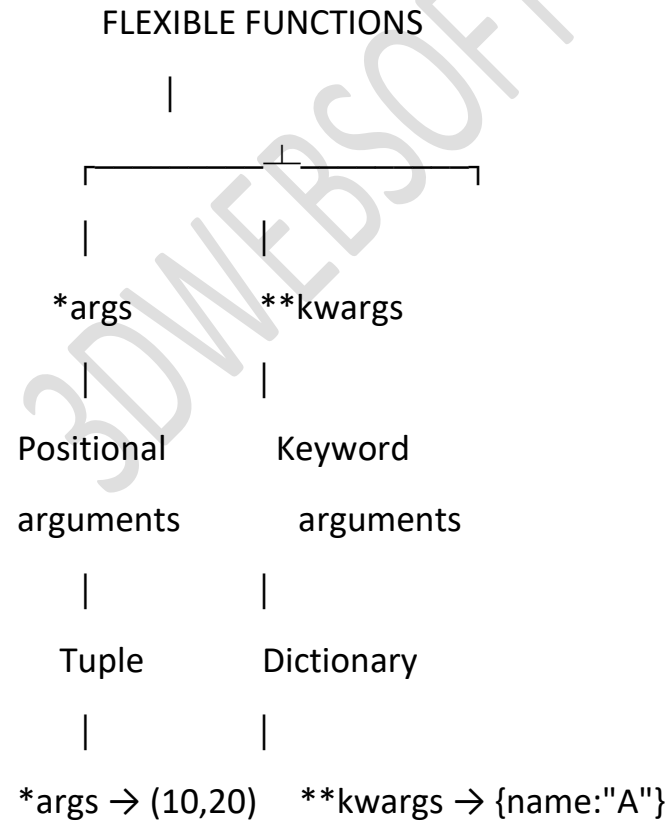
Additional Details:

class_name : 12th

section : A

city : Hyderabad

Quick Revision



3DWEBSOFT FOUNDATION

Remember:

***args = many positional values**

****kwargs = many named/keyword values**

Homework

1. What is a flexible function?
2. What is the purpose of *args?
3. What data type does args represent?
4. What is the purpose of **kwargs?
5. What data type does kwargs represent?
6. Write a function that accepts any number of numbers and finds their maximum.
7. Write a function that accepts student information using **kwargs.
8. Write a function using both *args and **kwargs.
9. Explain the difference between *args and **kwargs.
10. Create a **Flexible Shopping Cart** program using *args.

Challenge

Create a function:

```
def employee(name, *skills, **details):
    pass
```

It should accept:

- Employee name
- Any number of skills
- Any number of additional details

Example:

3DWEBSOFT FOUNDATION

```
employee(
    "Fathima",
    "Python",
    "Django",
    "SQL",
    experience=3,
    city="Hyderabad"
)
```

The program should display all the information neatly.

3DWEBSOFT FOUNDATION